Module 3 — Customization

Lesson 3.6

Output styles

Output styles change how Claude responds — brevity, tone, default verbosity. Useful in narrow situations; easy to overdo.

Why it matters

Most users never touch output styles and miss real wins (a "concise" style for review sessions, a "teaching" style when pairing with a junior). But it's also easy to over-customize and end up with a style that fights your work. This lesson covers the practical use cases.

What an output style controls

Output styles are presets that tweak Claude's default response shape. They typically affect:

- Length and verbosity
- Whether Claude explains its reasoning vs. just acts
- Whether code answers include surrounding commentary
- Tone (terse, conversational, instructional)

They do **not** change Claude's capabilities — same tools, same model. Just the shape of replies.

Built-in styles

Available styles depend on your version. Common ones:

Style	When to use
`default`	Most work
`concise`	Code review, long sessions where verbosity is friction
`learning` / `explanatory`	Pairing with someone learning the codebase or stack
`terse`	Headless / scripted contexts where you want minimal noise

Set via `/config` or `settings.json`:

```
{
  "outputStyle": "concise"
}
```

Or change mid-session with `/output-style <name>` (slash command name varies; check `/help`).

When to switch styles

Context	Style
Daily coding	`default`
Reviewing many small PRs	`concise`
Onboarding a new dev (pair session)	`learning`
Running in CI (capturing output)	`terse`
Writing docs	`default` or a custom one tuned for prose

Custom output styles

You can define your own. They're typically markdown files in `~/.claude/output-styles/` (path varies — confirm via docs). Each file is a system-prompt fragment that overrides defaults.

A minimal `concise-reviewer.md` might look like:

```
---
name: concise-reviewer
description: Punchy, no-frills code review tone
---

When reviewing code:
- Lead with the verdict (approve / request changes / blocker).
- List issues in priority order. Top 3 only.
- One sentence per issue. Cite file:line.
- No preamble. No restating of the diff. No "great work" closers.
```

Activate with `/output-style concise-reviewer`.

When NOT to bother

- **Don't tune a style to fix prompt problems.** If Claude's answers are wrong, the problem is the prompt or the context, not the style.
- **Don't switch styles every few turns.** It thrashes consistency. Pick one per session intent.
- **Don't ship a custom style without team buy-in.** It changes how Claude responds in shared sessions and can confuse teammates.

Try it

1. Run an interactive review session in `default`. Note how verbose the responses are.
2. Switch to `concise` (or write a custom `concise-reviewer` style) for the same kind of work.
3. Track over a session: did you read more or less? Catch more issues per minute?
4. If you don't see a measurable difference, stick with default. Styles are an optimization, not a requirement.

Gotchas

- **A heavy custom style can squash useful context.** "Always answer in 50 words" sounds great until Claude truncates a debugging walkthrough.
- **Styles don't change tool behavior.** Claude still calls Edit/Read/etc. the same way.
- **Per-session style overrides persist only for the session.** Set in `settings.json` to make permanent.
- **MCP/skill output is not always restyled.** Some structured outputs ignore the style preset.

Further reading

- Official output styles docs: https://docs.claude.com/en/docs/claude-code/output-styles

- 3.4 Custom slash commands — often a better tool for "always do X" than a style

Next

→ 4.1 Subagents overview